My_student_dsp_001_NB

My `student_dsp.py` notebook, first experimental version.

```
"""Fill the three required algorithms. common.py supplies fixtures and the
receiver."""
import numpy as np
import matplotlib.pyplot as plt
from scipy import signal
from common import FS, read_wav, relerr
import IPython.display as ipd
rng = np.random.default_rng()
```

```
%matplotlib widget
fsz = (7,4)
fsz2 = (fsz[0], 1.5*fsz[1]/2)
```

Let Lx be length of x, L length of blocks (frames), H hop size, Lp length of padding Lx Lp x |----------|---|

```
       L                                # of blocks: NB = 1 + ceil((Lx - L)/H)
```

blocks |-------| padding of x: Lp = (NB - 1)*H + L - Lx H |-------| H |-------| H |-------|

```
def analysis(x, L=1024, hop=None, window='hann', nfft=None):
    # TODO: periodic window, padded frames, local-index two-sided FFT,
metadata.
    """
    Break up x into M frames of length L using hop size `hop`
    (padding with zeros as necessary)
    Multiply each x frame by window w of length L
    Then take FFT of length nfft (default nfft=L) of windowed frames
    Return (M x L) array of STFTs and meta data: L, hop, nfft, window, w, Lx,
pad_front,
    pad_back, M, complex, fs
    """
    x = np.asarray(x)    # make sure x is an array
    cplx = np.iscomplexobj(x)   # check if x is complex-valued
    L = x.size if L>x.size else int(L)
    hop = L//4 if hop is None else int(hop)     # hop length
    nfft = L if nfft is None or nfft<L else int(nfft)     # FFT blocksize
nfft
    #pad_f = L     # front padding
    pad_f = 16     # front padding     # no front padding looks better in
spectrograms
                                       # but some padding is needed for NOLA
    xp = np.hstack((np.zeros(pad_f, x.dtype), x))
    #xp = np.hstack((np.zeros(pad_f, x.dtype), x, np.zeros(pad_f, x.dtype)))
    M = int(1 + np.ceil((xp.size-L)/hop))    # number of frames
```

```
    pad_b = int((M-1)*hop + L - xp.size)   # length of back padding
    xp = np.hstack((xp, np.zeros(pad_b, x.dtype)))
    w = signal.get_window(window, L, fftbins=True)   # periodic (vs
symmetric) window for DFT/FFT
    xA = np.zeros((M, L), x.dtype)
    for i in range(M):
        xA[i] = w*xp[i*hop: i*hop+L]
    if cplx:
        X = np.fft.fft(xA, n=nfft, axis=1)
    else:
        X = np.fft.rfft(xA, n=nfft, axis=1)
    meta = dict(L=L, hop=hop, N=nfft, window=window, w=w, Lx=x.size,
pad_front=pad_f,
                pad_back=pad_b, M=M, complex=cplx, fs=FS)
    return X, meta

x = np.arange(10)
X, meta = analysis(x, L=4, hop=2)
print(X)
print(meta)
```

```
[[ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 0.+0.j  0.+0.j  0.+0.j]
 [ 3.+0.j -2.+1.j  1.+0.j]
 [ 7.+0.j -4.+1.j  1.+0.j]
 [11.+0.j -6.+1.j  1.+0.j]
 [15.+0.j -8.+1.j  1.+0.j]]
{'L': 4, 'hop': 2, 'N': 4, 'window': 'hann', 'w': array([0. , 0.5, 1. ,
0.5]), 'Lx': 10, 'pad_front': 16, 'pad_back': 0, 'M': 12, 'complex': False,
'fs': 16000}
```

```
def frame_times(meta):
    """
    Frame center times in seconds, with the leading padding offset removed.
    """
    return (meta["hop"]*np.arange(meta["M"]) + meta["L"]/2 -
meta["pad_front"])/meta["fs"]


def frame_freqs(meta):
    """
    FFT bin frequencies in Hz: One-sided for real x, two-sided for complex x.
    """
    T = 1/meta["fs"]    # sampling time interval
    if meta["complex"]:
```

```
        ff = np.fft.fftfreq(meta["N"], T)
    else:
        ff = np.fft.rfftfreq(meta["N"], T)
        #ff = np.fft.fftfreq(meta["N"], T)
    return ff


frame_freqs(meta)
```

```
array([   0., 4000., 8000.])
```

```
# Convert STFTs to dB
def spec_db(X, floor=-80):
    m = np.abs(X); m = m / m.max()
    return np.maximum(20 * np.log10(m + 1e-12), floor)

# Read test signal
f_name = 'signals/tf_probe.wav'
_, x = read_wav(f_name)

ipd.Audio(x, rate=FS)
```

```
<IPython.lib.display.Audio object>
```

```
def show(ax, X, meta, title, fmax=4000):
    f = frame_freqs(meta); t = frame_times(meta)
    im = ax.pcolormesh(t, f, spec_db(X).T, vmin=-80, vmax=0, shading="auto")
    #im = ax.pcolormesh(t, f, spec_db(X).T, cmap='inferno', vmin=-80, vmax=0,
shading="auto")
    ax.set_ylim(0, fmax); ax.set_xlabel("Time (s)"); ax.set_ylabel("Frequency
(Hz)")
    ax.set_title(f"{title}\n fs={meta['fs']} Hz, {meta['window']},
L={meta['L']}, H={meta['hop']}, N={meta['N']}", fontsize=8)
    return im

def analysis_AI(x, L=1024, hop=256, window='hann', nfft=None):
    nfft = L if nfft is None else nfft
    if not (0 < hop <= L <= nfft):
        raise ValueError('Require 0 < hop <= L <= nfft')
    w = signal.get_window(window, L, fftbins=True)
    # Padding before sample zero covers window endpoints; crop precisely at
synthesis.
    left = L
    xp = np.pad(x, (left, L))
    count = 1 + int(np.ceil((len(xp)-L)/hop))
    needed = (count-1)*hop + L
    pad_b = needed-len(xp)
    xp = np.pad(xp, (0, needed-len(xp)))
    frames = np.array([xp[m*hop:m*hop+L]*w for m in range(count)])
    X = np.fft.fft(frames, n=nfft, axis=1)
    #meta = dict(w=w, L=L, hop=hop, nfft=nfft, left=left, length=len(x),
```

```
    #            real_input=not np.iscomplexobj(x))
    meta = dict(L=L, hop=hop, N=nfft, window=window, w=w, Lx=x.size,
pad_front=L,
                pad_back=pad_b, M=count, complex=np.iscomplexobj(x), fs=FS)
    return X, meta

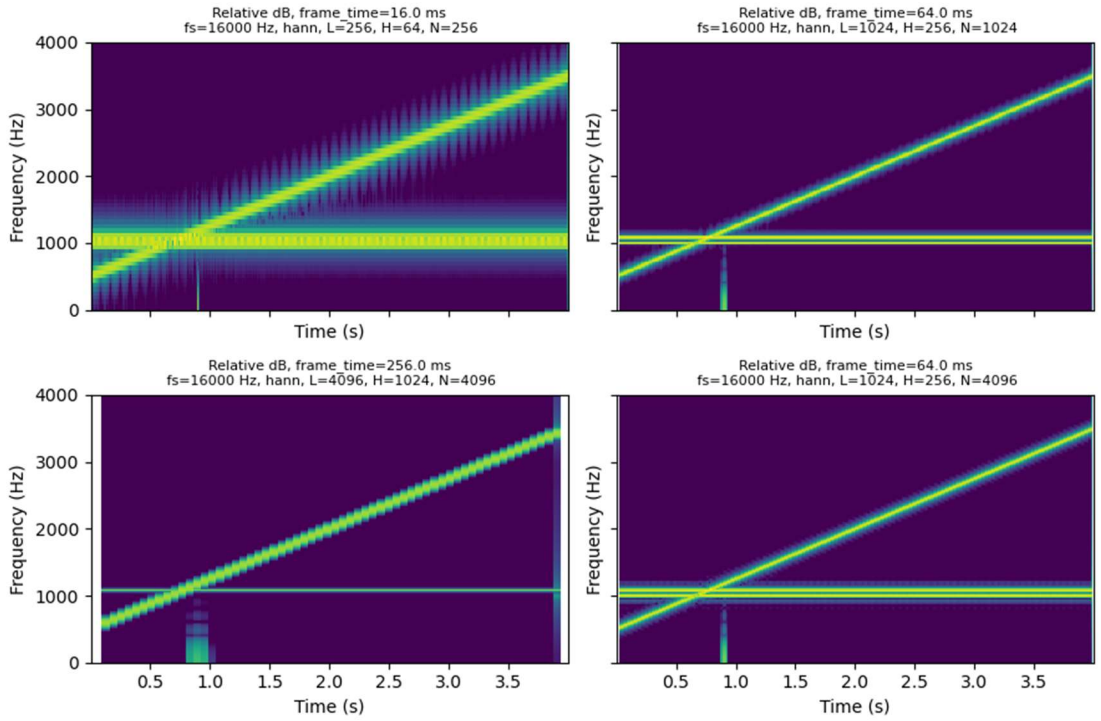
x_test = np.arange(10)
L_test = 4
left = L_test
xp_test = np.pad(x_test, (left, L_test))
print(xp_test)
```

```
[0 0 0 0 0 1 2 3 4 5 6 7 8 9 0 0 0 0]
```

```
# ------------ Part A1 ------------------------------------------------------
-----------------

# Spectrograms for part A1
L1, L2, L3, L4 = 256, 1024, 4096, 1024
N1, N2, N3, N4 = L1, L2, L3, 4096
fig, axs = plt.subplots(2, 2, figsize=(9, 6), sharey=True, sharex=True)
fig.canvas.toolbar_position = 'top'
X, meta = analysis(x, L=L1, hop=L1//4, nfft=N1)
print(X.shape)
im = show(axs[0,0], X, meta, f"Relative dB, frame_time={1e3*L1/meta["fs"]}
ms")
X, meta = analysis(x, L=L2, hop=L2//4, nfft=N2)
im = show(axs[0,1], X, meta, f"Relative dB, frame_time={1e3*L2/meta["fs"]}
ms")
#X, meta = analysis_AI(x, L=L1, hop=L1//4, nfft=N1)
#im = show(axs[0,1], X[:,:L1//2+1], meta, "Relative dB")
X, meta = analysis(x, L=L3, hop=L3//4, nfft=N3)
im = show(axs[1,0], X, meta, f"Relative dB, frame_time={1e3*L3/meta["fs"]}
ms")
X, meta = analysis(x, L=L4, hop=L4//4, nfft=N4)
im = show(axs[1,1], X, meta, f"Relative dB, frame_time={1e3*L4/meta["fs"]}
ms")
#X, meta = analysis_AI(x, L=L3, hop=L3//4, nfft=N3)
#im = show(axs[1,1], X[:,:L3//2+1], meta, "Relative dB")
plt.tight_layout()
```

```
(998, 129)
```




```
# Plot frequency resolution
L1, L2, L3, L4 = 256, 1024, 4096, 1024
N1, N2, N3, N4 = L1, L2, L3, 4096
fig, axs = plt.subplots(2, 2, figsize=(9, 6), sharey=True, sharex=True)
fig.canvas.toolbar_position = 'top'

X, meta = analysis(x, L=L1, hop=L1//4, nfft=N1)
f = frame_freqs(meta); t = frame_times(meta)
ixt = np.where(t<=3.5)[0]; ix = ixt[-1]
#print(t[ix])    # measurement time used
axs[0,0].plot(f, 20*np.log10(np.abs(X[ix])))
axs[0,0].set_title(f'L={L1}, nfft={N1}, tmeas={t[ix]:.1f} sec')
axs[0,0].set_ylabel('|X(f)| (dB)')
axs[0,0].grid(alpha=0.5)
axs[0,0].set_ylim([-80, 60])
axs[0,0].set_xlim([600, 1400])

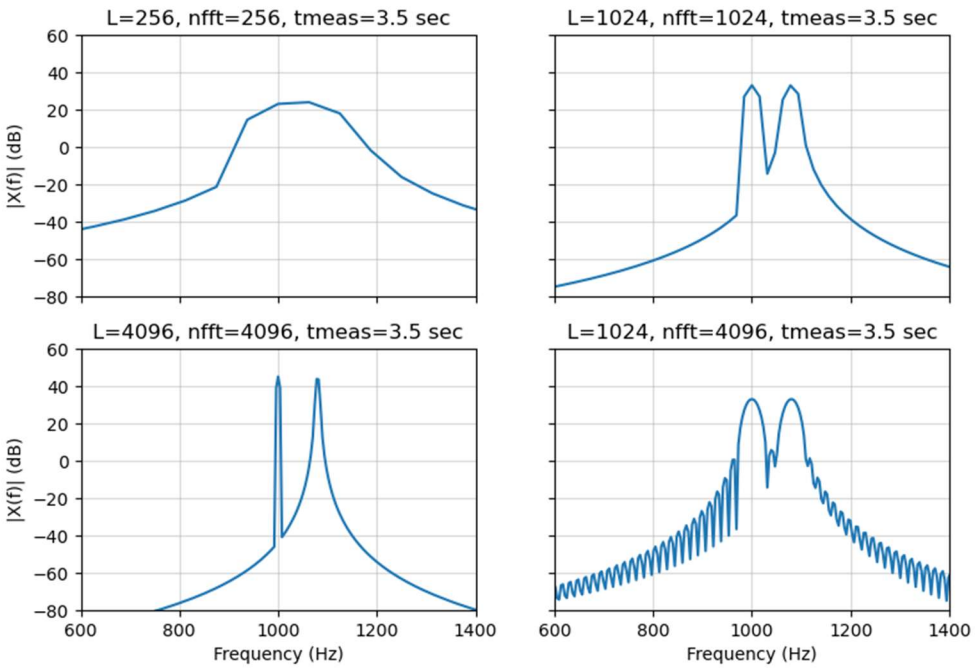
X, meta = analysis(x, L=L2, hop=L2//4, nfft=N2)
f = frame_freqs(meta); t = frame_times(meta)
ixt = np.where(t<=3.5)[0]; ix = ixt[-1]
#print(t[ix])    # measurement time used
axs[0,1].plot(f, 20*np.log10(np.abs(X[ix])))
axs[0,1].set_title(f'L={L2}, nfft={N2}, tmeas={t[ix]:.1f} sec')
axs[0,1].grid(alpha=0.5)

X, meta = analysis(x, L=L3, hop=L3//4, nfft=N3)
f = frame_freqs(meta); t = frame_times(meta)
```

```
ixt = np.where(t<=3.5)[0]; ix = ixt[-1]
#print(t[ix])    # measurement time used
axs[1,0].plot(f, 20*np.log10(np.abs(X[ix])))
axs[1,0].set_title(f'L={L3}, nfft={N3}, tmeas={t[ix]:.1f} sec')
axs[1,0].set_ylabel('|X(f)| (dB)')
axs[1,0].set_xlabel('Frequency (Hz)')
axs[1,0].grid(alpha=0.5)

X, meta = analysis(x, L=L4, hop=L4//4, nfft=N4)
f = frame_freqs(meta); t = frame_times(meta)
ixt = np.where(t<=3.5)[0]; ix = ixt[-1]
#print(t[ix])    # measurement time used
axs[1,1].plot(f, 20*np.log10(np.abs(X[ix])))
axs[1,1].set_title(f'L={L4}, nfft={N4}, tmeas={t[ix]:.1f} sec')
axs[1,1].set_xlabel('Frequency (Hz)')
axs[1,1].grid(alpha=0.5)

plt.show()
```




```
# Plot transient width
L1, L2, L3, L4 = 256, 1024, 4096, 1024
N1, N2, N3, N4 = L1, L2, L3, 4096
fig, axs = plt.subplots(2, 2, figsize=(9, 6), sharey=True, sharex=True)
fig.canvas.toolbar_position = 'top'

X, meta = analysis(x, L=L1, hop=L1//4, nfft=N1)
f = frame_freqs(meta); t = frame_times(meta)
```

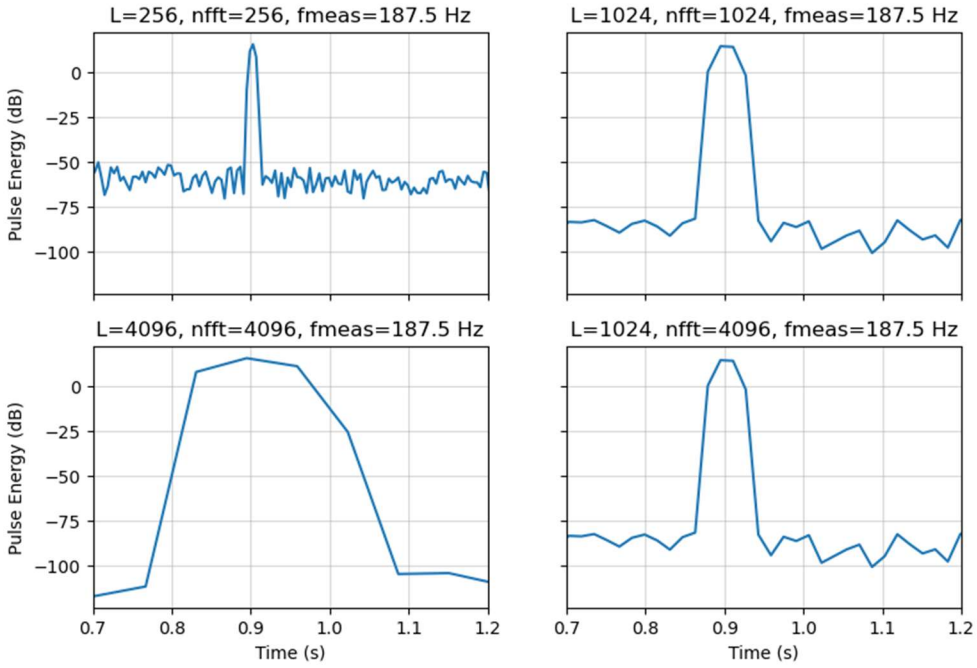
```
ixf = np.where(f<500)[0]; ix = ixf[-5]
#print(f[ix])    # measurement frequency used
axs[0,0].plot(t, 20*np.log10(np.abs(X[:,ix])))
axs[0,0].set_title(f'L={L1}, nfft={N1}, fmeas={f[ix]} Hz')
axs[0,0].set_ylabel('Pulse Energy (dB)')
axs[0,0].grid(alpha=0.5)
axs[0,0].set_xlim([0.7, 1.2])

X, meta = analysis(x, L=L2, hop=L2//4, nfft=N2)
f = frame_freqs(meta); t = frame_times(meta)
ixf = np.where(f<500)[0]; ix = ixf[-20]
#print(f[ix])    # measurement frequency used
axs[0,1].plot(t, 20*np.log10(np.abs(X[:,ix])))
axs[0,1].set_title(f'L={L2}, nfft={N2}, fmeas={f[ix]} Hz')
axs[0,1].grid(alpha=0.5)

X, meta = analysis(x, L=L3, hop=L3//4, nfft=N3)
f = frame_freqs(meta); t = frame_times(meta)
ixf = np.where(f<500)[0]; ix = ixf[-80]
#print(f[ix])    # measurement frequency used
axs[1,0].plot(t, 20*np.log10(np.abs(X[:,ix])))
axs[1,0].set_title(f'L={L3}, nfft={N3}, fmeas={f[ix]} Hz')
axs[1,0].set_ylabel('Pulse Energy (dB)')
axs[1,0].set_xlabel('Time (s)')
axs[1,0].grid(alpha=0.5)

X, meta = analysis(x, L=L4, hop=L4//4, nfft=N4)
f = frame_freqs(meta); t = frame_times(meta)
ixf = np.where(f<500)[0]; ix = ixf[-80]
#print(f[ix])    # measurement frequency used
axs[1,1].plot(t, 20*np.log10(np.abs(X[:,ix])))
axs[1,1].set_title(f'L={L4}, nfft={N4}, fmeas={f[ix]} Hz')
axs[1,1].set_xlabel('Time (s)')
axs[1,1].grid(alpha=0.5)

plt.show()
```



With L=256 the frequency difference of 80 Hz cannot be resolved. Transient pulsewidth defined as -10 dB points from maximum pulse energy.

L	Nfft	Window ms	Bin Hz	Main lobe Hann Hz	Hop ms	Pwidth ms
256	256	16.0	62.5	250.0	4.0	9
1024	1024	64.0	15.63	62.5	16.0	37
4096	4096	256.0	3.906	15.625	64.0	139
1024	4096	256.0	3.906	62.5	16.0	37

```
# ------------ Part A2 ------------------------------------------------------
--------------------
```

```
# Read test signal
f_name2 = 'signals/leakage.wav'   # 1024 samples with strong 1003 Hz and weak
1140 Hz tones
_, x_leak = read_wav(f_name2)
```

```
ipd.Audio(x_leak, rate=FS)
```

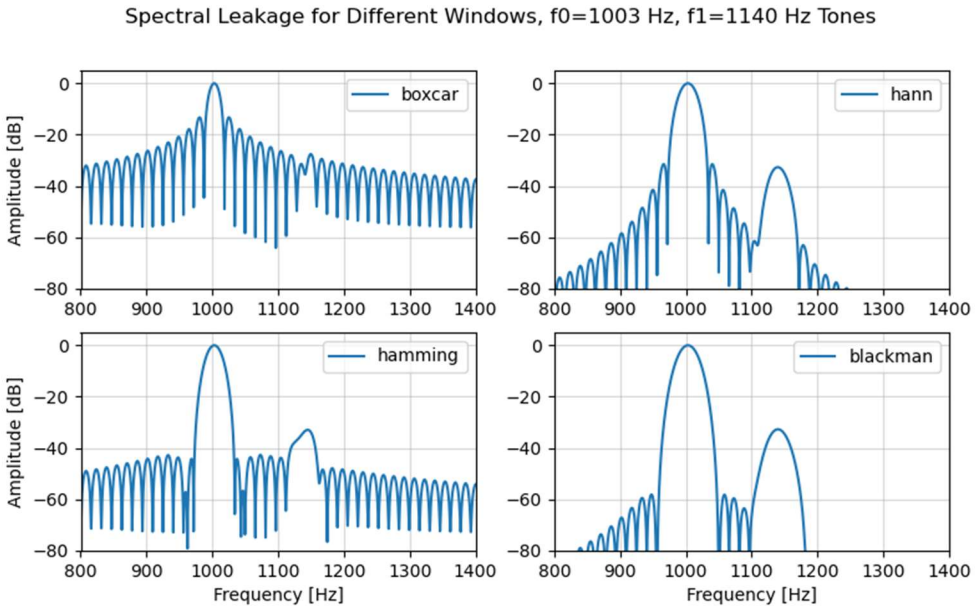
```
<IPython.lib.display.Audio object>
```

```
# A2 Spectral leakage
L2, N2 = 1024, 65536
nn2 = np.arange(L2)
```

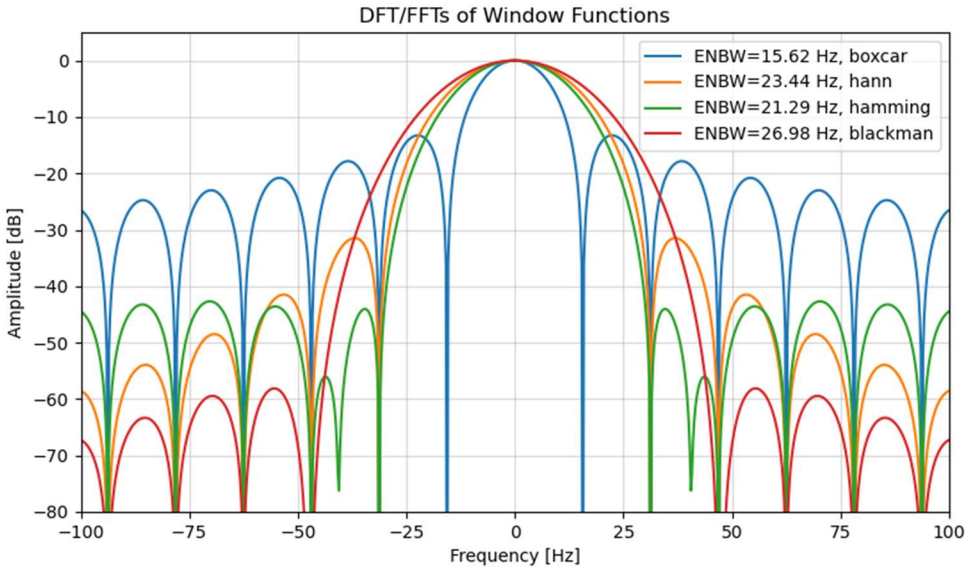
```
win_names = ['boxcar', 'hann', 'hamming', 'blackman']
w2 = np.zeros((len(win_names), L2))   # window, time domain
ENBW = np.zeros(len(win_names))
W2 = np.zeros((len(win_names), N2))    # window, frequency domain
X_leak = np.zeros((len(win_names), N2//2+1))     # 'leakage' signal,
frequency domian
for i in range(len(win_names)):
    w2[i] = signal.get_window(win_names[i], L2, fftbins=True)   # time domain
window
    ENBW[i] = FS*np.sum(w2[i]**2)/np.sum(w2[i])**2
    W2[i] = np.abs(np.fft.fft(w2[i], N2))    # FFT of window
    W2[i] = W2[i]/W2[i,0]    # Window FFT normalized by dc amplitude
    X_leak[i] = np.abs(np.fft.rfft(x_leak*w2[i], N2))/w2[i].sum()   # signal
in freq domain, normalized by window sum

# Results from spectral leakage
ff2 = np.fft.rfftfreq(N2, 1/FS)
A20, A21 = -80, 5    # amplitude limits in dB
f20, f21 = 800, 1400    # frequency limits in Hz
fig, axs = plt.subplots(2, 2, figsize=(9, 5))
fig.canvas.toolbar_position = 'top'
fig.suptitle(f'Spectral Leakage for Different Windows, f0=1003 Hz, f1=1140 Hz
Tones')
#axs[0,0].plot(ff2, X_leak[0], label=win_names[0])
axs[0,0].plot(ff2, 20*np.log10(2/.65*X_leak[0]), label=win_names[0])
axs[0,0].set_ylabel('Amplitude [dB]')
axs[0,0].grid(alpha=0.5)
axs[0,0].set_ylim([A20, A21])
axs[0,0].set_xlim([f20, f21])
axs[0,0].legend()
#axs[0,1].plot(ff2, X_leak[1], label=win_names[1])
axs[0,1].plot(ff2, 20*np.log10(2/.65*X_leak[1]), label=win_names[1])
axs[0,1].grid(alpha=0.5)
axs[0,1].set_ylim([A20, A21])
axs[0,1].set_xlim([f20, f21])
axs[0,1].legend()
#axs[1,0].plot(ff2, X_leak[2], label=win_names[2])
axs[1,0].plot(ff2, 20*np.log10(2/.65*X_leak[2]), label=win_names[2])
axs[1,0].set_ylabel('Amplitude [dB]')
axs[1,0].set_xlabel('Frequency [Hz]')
axs[1,0].grid(alpha=0.5)
axs[1,0].set_ylim([A20, A21])
axs[1,0].set_xlim([f20, f21])
axs[1,0].legend()
#axs[1,0].plot(ff2, X_leak[3], label=win_names[3])
axs[1,1].plot(ff2, 20*np.log10(2/.65*X_leak[3]), label=win_names[3])
axs[1,1].set_xlabel('Frequency [Hz]')
axs[1,1].grid(alpha=0.5)
axs[1,1].set_ylim([A20, A21])
axs[1,1].set_xlim([f20, f21])
```

```
axs[1,1].legend()
plt.show()
```




```
# Plot of window FFTs
ff2w = np.fft.fftfreq(N2, 1/FS)
#ff2w = np.fft.fftshift(np.fft.fftfreq(N2, 1/FS))
A20w, A21w = -80, 5    # amplitude limits in dB
f20w, f21w = -100, 100    # frequency limits in Hz
fig, axs = plt.subplots(1, 1, figsize=(9, 5))
fig.canvas.toolbar_position = 'top'
for i in range(len(win_names)):
    axs.plot(ff2w, 20*np.log10(np.maximum(W2[i],1e-6)),
label=f'ENBW={ENBW[i]:2.2f} Hz, {win_names[i]}')
axs.set_title('DFT/FFTs of Window Functions')
axs.set_ylabel('Amplitude [dB]')
axs.set_xlabel('Frequency [Hz]')
plt.grid(alpha=0.5)
axs.set_ylim([A20w, A21w])
axs.set_xlim([f20w, f21w])
axs.legend()
plt.show()
```



Here we use L=1024, Nfft=65,536, FS=16,000 Hz

Main lobe width for Boxcar is 2xFS/L, for Hann and Hamming it is 4xFS/L, for Blackman it is 6xFS/L

Parameter	Boxcar	Hann	Hamming	Blackman
null2null	31.25 Hz	62.5 Hz	62.5 Hz	93.75 Hz
pksidelobe	-13.26 dB	-31.47 dB	-42.67 dB	-58.11 dB
ENBW	15.625 Hz	23.438 Hz	21.294 Hz	26.981 Hz

```
# ----------- Part B1 ----------------------------------------------------------
----------------------

def synthesis(X, meta):
    """
    X is (M x L) array of STFTs and meta is the accompanying dictionary of
meta data.
    `synthesis` takes inverse DFT/FFT to obtain time domain frames, then
windows each
    time domain frame using meta["w"] window (this is beneficial if X has
been modified
    between `analysis` and `synthesis`). Samplewise normalization to undo the
scaling
    introduced by the `analysis` and `synthesis` windows needs to be applied
in the
    denominator d while reconstructing the output signal y from the frames. A
nonzero
    denominator for every retained sample is necessary for this to work
correctly (NOLA:
    nonzero overlap add requirement).
    Returns y, d  (output y, denominator d, cropped to length of original
```

```
signal).
    """
    L = meta['L']     # frame length
    hop = meta['hop']     # hop length in samples
    w = meta['w']     # window samples (length L)
    nfft = meta['N']    # DFT/FFT block size
    if meta['complex']:
        frames = np.fft.ifft(X, n=nfft, axis=1)[:, :L]   # recovered frames
(complex-valued)
    else:
        frames = np.fft.irfft(X, n=nfft, axis=1)[:, :L]   # recovered frames
(real-valued)
    M = frames.shape[0]      # number of frames
    Lyp = (M - 1)*hop + L     # length of padded output y
    yp = np.zeros(Lyp, frames.dtype)    # allocate array for yp
    dp = np.zeros(Lyp)       # allocate array for dp (padded denominator)
    for i in range(M):
        idx = slice(i*hop, i*hop+L)    # current range of indexes
        yp[idx] += frames[i]*w    # windowed frame
        dp[idx] += w*w         # samplewise denominator for normalization
    crop = slice(meta['pad_front'], meta['pad_front']+meta['Lx'])
    d = dp[crop]       # denominator for original data length
    if np.any(np.abs(d) < 1e-12):
        raise ValueError('NOLA violated in retained samples')
    y = yp[crop]/d     # samplewise normalized output with original data
length
    if not meta['complex']:
        y = y.real
    return y, d    # output and denominator


# Read jammed test signal
f_name3 = 'signals/jammed.wav'
_, x_jam = read_wav(f_name3)

ipd.Audio(x_jam, rate=FS)

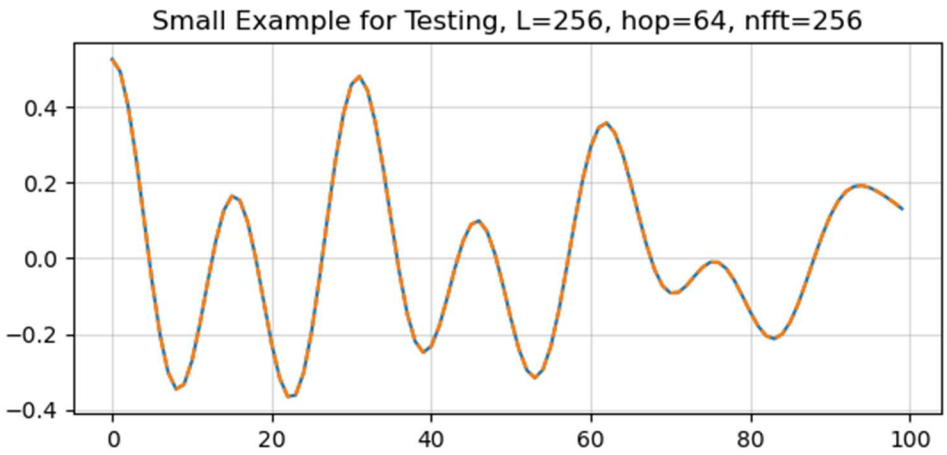
<IPython.lib.display.Audio object>

# Small example for testing
X, meta = analysis(x, L=L1, hop=L1//4, nfft=N1)
x_hat, d = synthesis(X, meta)
x_err = x - x_hat
print(np.min(np.abs(d)))
print(np.max(np.abs(x_err)))

0.0014485813926750633
8.326672684688674e-16

fig, axs = plt.subplots(1, 1, figsize=(7,3))
fig.canvas.toolbar_position = 'top'
```

```
axs.plot(x[:100])
axs.plot(x_hat[:100], linestyle='--')
axs.set_title(f'Small Example for Testing, L={L1}, hop={L1//4}, nfft={N1}')
axs.grid(alpha=0.5)
plt.show()
```




```
# Reconstruction using normalized synthesis
L = 1024
h1, h2, h3, h4, h5 = L, 3*L/4, L/2, L/4, L/8
w1, w2 = 'boxcar', 'hann'

X, meta = analysis(x, L=L, hop=h1, window=w1, nfft=L)
x_hat, d = synthesis(X, meta)
x_err = x - x_hat
print(f'hop={h1}, frames={X.shape[0]}, window={w1}, Drange={np.min(d)} to
{np.max(d)}, error={np.max(np.abs(x_err)):.1e}')

try:
    X, meta = analysis(x, L=L, hop=h1, window=w2, nfft=L)
    x_hat, d = synthesis(X, meta)
    x_err = x - x_hat
    print(f'hop={h1}, frames={X.shape[0]}, window={w2}, Drange={np.min(d)} to
{np.max(d)}, error={np.max(np.abs(x_err)):.1e}')
except:
    print(f'hop={h1}, window={w2}, NOLA violated')

X, meta = analysis(x, L=L, hop=h2, window=w2, nfft=L)
x_hat, d = synthesis(X, meta)
x_err = x - x_hat
print(f'hop={h2}, frames={X.shape[0]}, window={w2}, Drange={np.min(d):.2e} to
{np.max(d)}, error={np.max(np.abs(x_err)):.1e}')

X, meta = analysis(x, L=L, hop=h3, window=w2, nfft=L)
x_hat, d = synthesis(X, meta)
x_err = x - x_hat
```

```
print(f'hop={h3}, frames={X.shape[0]}, window={w2}, Drange={np.min(d):.2e} to
{np.max(d)}, error={np.max(np.abs(x_err)):.1e}')

X, meta = analysis(x, L=L, hop=h4, window=w2, nfft=L)
x_hat, d = synthesis(X, meta)
x_err = x - x_hat
print(f'hop={h4}, frames={X.shape[0]}, window={w2}, Drange={np.min(d):.2e} to
{np.max(d):.1f}, error={np.max(np.abs(x_err)):.1e}')

X, meta = analysis(x, L=L, hop=h5, window=w2, nfft=L)
x_hat, d = synthesis(X, meta)
x_err = x - x_hat
print(f'hop={h5}, frames={X.shape[0]}, window={w2}, Drange={np.min(d):.2e} to
{np.max(d):.1f}, error={np.max(np.abs(x_err)):.1e}')
```

```
hop=1024, frames=63, window=boxcar, Drange=1.0 to 1.0, error=2.9e-16
hop=1024, window=hann, NOLA violated
hop=768.0, frames=84, window=hann, Drange=5.80e-06 to 1.0, error=3.0e-14
hop=512.0, frames=125, window=hann, Drange=5.80e-06 to 1.0, error=3.0e-14
hop=256.0, frames=248, window=hann, Drange=5.80e-06 to 1.5, error=3.0e-14
hop=128.0, frames=494, window=hann, Drange=5.80e-06 to 3.0, error=3.0e-14
```

Ovlp %	Hop	Frames	Window	Denominator range	Error
0	1024	63	boxcar	1.0 to 1.0	2.9e-14
0	1024	63	hann	failure	NOLA
25	768	84	hann	5.8e-06 to 1.0	3.0e-14
50	512	125	hann	5.8e-06 to 1.0	3.0e-14
75	256	248	hann	5.8e-06 to 1.5	3.0e-14
87.5	128	494	hann	5.8e-06 to 3.0	3.0e-14

```
# --------- Part B2 ----------------------------------------------------------
----------------------

def ola_filter(x, h, B=512, nfft=None):
    """
    Exact linear convolution using overlap-add method and FFT multiplication.
    Output length len(x)+len(h)-1 (full convolution).
    x is broken up into blocks of size B
    """
    x = np.asarray(x)   # data sequence
    h = np.asarray(h)   # filter impulse response
    Lx, Lh = x.size, h.size
    if nfft is None:
        nfft = 1 << (B+Lh-1).bit_length()
    if nfft < B+Lh-1:
        nfft = B+Lh-1
    cplx = np.iscomplexobj(x) or np.iscomplexobj(h)
    H = np.fft.fft(h, nfft)     # FFT of filter response h
```

```
    y = np.zeros(Lx+Lh-1, complex)   # output array
    i = 0
    for n in range(0, Lx, B):
        blk = x[n:n+B]    # block of length B
        #blk_conv = np.fft.ifft(np.fft.fft(blk, nfft) * H, nfft)[:B+Lh-1]  #
block convolution
        blk_conv = np.fft.ifft(np.fft.fft(blk, nfft) * H, nfft)  # block
convolution
        Lconv = len(blk)+Lh-1
        y[n:n+Lconv] += blk_conv[:Lconv]   # overlap-add
        i += 1
    print(i)    # number of frames
    if not cplx:
        y = y.real
    return y


# Testing with small B
B_test = 4
x_test = np.arange(10)
for nn in range(0, len(x_test), B_test):
    print(x_test[nn:nn+B_test])
```

```
[0 1 2 3]
[4 5 6 7]
[8 9]
```

```
# Read jammed test signal
f_name3 = 'signals/jammed.wav'
_, x_jam = read_wav(f_name3)
```

```
ipd.Audio(x_jam, rate=FS)
```

```
<IPython.lib.display.Audio object>
```

```
# B2. Frequency domain FIR filtering
B_jam = [512, 128, 2048]
nfft_jam = [1024, 256, 4096]
h = signal.firwin(129, 1200, fs=FS, window="hamming")
y_jam = ola_filter(x_jam, h, B=B_jam[2])
reference = np.convolve(x_jam, h)
assert len(y_jam) == len(reference)
assert relerr(reference, y_jam) < 1e-10
# TODO: repeat for B=128 and 2048.
```

```
32
```

```
# Test audio result
ipd.Audio(y_jam, rate=FS)
```

```
<IPython.lib.display.Audio object>
```

```
# Filter with different B and report errors
y_jam = ola_filter(x_jam, h, B=B_jam[1], nfft=nfft_jam[1])
print(f'\nB={B_jam[1]}, nfft={nfft_jam[1]}')
print(f'L(x_jam)={len(x_jam)}, L(y_jam)={len(y_jam)},
L(ref)={len(reference)}')
print(f'Error: {np.max(np.abs(y_jam-reference))}')
print(f'relerr: {relerr(reference, y_jam)}')

y_jam = ola_filter(x_jam, h, B=B_jam[0], nfft=nfft_jam[0])
print(f'\nB={B_jam[0]}, nfft={nfft_jam[0]}')
print(f'L(x_jam)={len(x_jam)}, L(y_jam)={len(y_jam)},
L(ref)={len(reference)}')
print(f'Error: {np.max(np.abs(y_jam-reference))}')
print(f'relerr: {relerr(reference, y_jam)}')

y_jam = ola_filter(x_jam, h, B=B_jam[2], nfft=nfft_jam[2])
print(f'\nB={B_jam[2]}, nfft={nfft_jam[2]}')
print(f'L(x_jam)={len(x_jam)}, L(y_jam)={len(y_jam)},
L(ref)={len(reference)}')
print(f'Error: {np.max(np.abs(y_jam-reference))}')
print(f'relerr: {relerr(reference, y_jam)}')
```

```
500

B=128, nfft=256
L(x_jam)=64000, L(y_jam)=64128, L(ref)=64128
Error: 2.7755575615628914e-16
relerr: 3.4301494526076115e-16
125

B=512, nfft=1024
L(x_jam)=64000, L(y_jam)=64128, L(ref)=64128
Error: 2.220446049250313e-16
relerr: 3.2815413012810876e-16
32

B=2048, nfft=4096
L(x_jam)=64000, L(y_jam)=64128, L(ref)=64128
Error: 2.220446049250313e-16
relerr: 3.4199731608199134e-16
```

Block size B	FFT size	# Blocks	Relative Error
128	256	500	3.43e-16
512	1024	125	3.28e-16
2048	4096	32	3.42e-16

```
# ----------- Part C2 ------------------------------------------------------
-------------------------
```

```
from pathlib import Path
from common import fixtures
a = fixtures(Path("signals"))

# QPSK receiver (from common.py)
def receive(x, symbols, bits, pulse):
    """Known timing, no carrier recovery or per-symbol correction. Transients
excluded."""
    mf = np.convolve(x, pulse.conj()[::-1])
    z = mf[(len(pulse)-1)+16*np.arange(len(symbols))]
    keep = slice(10, -10); z = z[keep]; ref = symbols[keep]; b = bits[keep]
    evm = 100*np.linalg.norm(z-ref)/np.linalg.norm(ref)
    decisions = np.column_stack((z.real < 0, z.imag < 0))
    ber = np.mean(decisions != b)
    return dict(evm_percent=float(evm), ber=float(ber)), z

# Read QPSK signal
x_qpsk = a["tx"]

ipd.Audio([x_qpsk.real, x_qpsk.imag], rate=FS)
```

```
<IPython.lib.display.Audio object>
```

```
# Generate spectrogram
X, meta = analysis(x_qpsk)

# Generate different signal variants
y_rec, d_rec = synthesis(X, meta)
y_mag, d_mag = synthesis(np.abs(X).astype(complex), meta)
y_randth, d_randth = synthesis(X*np.exp(2j*np.pi*rng.random((X.shape[0],
1))), meta)

# Receive different QPSK signals
a = fixtures(Path("signals"))
met_orig, z_orig = receive(x_qpsk, a["symbols"], a["bits"], a["pulse"])
met_rec, z_rec = receive(y_rec, a["symbols"], a["bits"], a["pulse"])
met_mag, z_mag = receive(y_mag, a["symbols"], a["bits"], a["pulse"])
met_randth, z_randth = receive(y_randth, a["symbols"], a["bits"], a["pulse"])

# Plot signal constellations
fig, axs = plt.subplots(2, 2, figsize=(8, 6), sharey=True, sharex=True)
fig.canvas.toolbar_position = 'top'
fig.suptitle(f'QPSK Phase Coherent Communication')

axs[0,0].plot(z_orig.real, z_orig.imag, '.',
label=f'EVM={met_orig['evm_percent']:.1f}%')
axs[0,0].set_title(f'Original, BER={met_orig['ber']}')
axs[0,0].set_ylabel('Q')
#axs[0,0].set_xlabel('I')
axs[0,0].grid(alpha=0.5)
axs[0,0].legend()
```

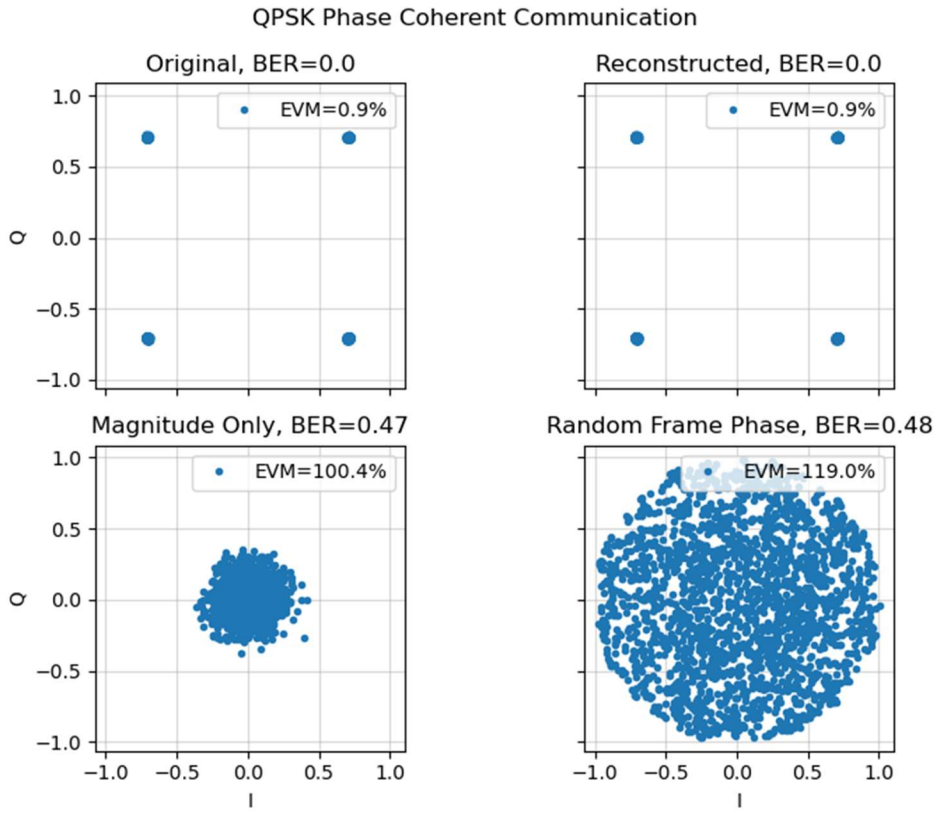
```
axs[0,0].set_aspect('equal')

axs[0,1].plot(z_rec.real, z_rec.imag, '.',
label=f'EVM={met_rec['evm_percent']:.1f}%')
axs[0,1].set_title(f'Reconstructed, BER={met_rec['ber']}')
#axs[0,1].set_ylabel('Q')
#axs[0,1].set_xlabel('I')
axs[0,1].grid(alpha=0.5)
axs[0,1].legend()
axs[0,1].set_aspect('equal')

axs[1,0].plot(z_mag.real, z_mag.imag, '.',
label=f'EVM={met_mag['evm_percent']:.1f}%')
axs[1,0].set_title(f'Magnitude Only, BER={met_mag['ber']:.2f}')
axs[1,0].set_ylabel('Q')
axs[1,0].set_xlabel('I')
axs[1,0].grid(alpha=0.5)
axs[1,0].legend()
axs[1,0].set_aspect('equal')

axs[1,1].plot(z_randth.real, z_randth.imag, '.',
label=f'EVM={met_randth['evm_percent']:.1f}%')
axs[1,1].set_title(f'Random Frame Phase, BER={met_randth['ber']:.2f}')
#axs[1,1].set_ylabel('Q')
axs[1,1].set_xlabel('I')
axs[1,1].grid(alpha=0.5)
axs[1,1].legend()
axs[1,1].set_aspect('equal')

plt.tight_layout()
plt.show()
```


QPSK Phase Coherent Communication
Original, BER=0.0
EVM=0.9%
Reconstructed, BER=0.0
EVM=0.9%
Magnitude Only, BER=0.47
EVM=100.4%
Random Frame Phase, BER=0.48
EVM=119.0%
Q
I
1.0
0.5
0.0
−0.5
−1.0